A Hybrid Rule-Driven and LLM-Based Agent for Automated Compliance Verification of Academic Development Plans

Kevin Beltrán Martínez
Code: 2220221003
Systems Engineering
Faculty of Engineering
University of Ibagué
Ibagué, Colombia

Jeryleefth Lasso Motta
Code: 2220231074
Systems Engineering
Faculty of Engineering
University of Ibagué
Ibagué, Colombia

Alexandra La Cruz
Professor
alexandra.lacruz@unibague.edu.co
Faculty of Engineering
University of Ibagué
Ibagué, Colombia

***Abstract*—Academic program offices at the University of Ibagué manually verify, every semester, that more than twenty Academic Development Plans (PDA from the Spanish acronym) comply with institutional guidelines, ABET student outcomes, and course-specific competency mappings. This paper presents UnibaBot PDA, an intelligent agent that automates this review using a hybrid pipeline combining deterministic rules with LLM-based evidence extraction and validation. The LLM acts as a structured extractor; every compliance verdict is computed in code. Evaluated on a gold dataset of 106 entries across six PDAs (51 training entries and 55 hold-out test entries from courses never seen during development), the final system reaches perfect classification performance, with precision and recall of 1.000 on the non-compliant class and full rule coverage. Fine-tuning of the LLM is not included in the final system, as preliminary experiments at this data scale showed no measurable benefit over the architecture-first approach.**

***Index Terms*—large language models, retrieval-augmented generation, compliance verification, ABET, rule-driven matching, academic quality assurance, intelligent agents**

I. Introduction

Every semester, the offices of each academic program at the University of Ibagué manually review more than twenty Academic Development Plans (PDAs) against institutional guidelines and the improvement directives for every period. The curricular leader (a faculty member from the program), the reviewer, reads each document, compares it against a set of rules, records any problems they identify, and reports to the faculty member owner of the document. In the absence of a standardization mechanism, errors go undetected, different reviewers apply the criteria inconsistently with the institutional guidelines, and the process is time-consuming for the reviewer. These problems compromise both review quality and the time for the reviewer, preventing burnout.

PDAs follow a semi-standardized template with sections for learning outcomes, assessment methods, course schedule, and improvement actions from the previous semester. The rules they must follow form a structured set of connected requirements: seven ABET student outcomes (O1–O7) with specific performance indicators, three program-level competencies (C1–C3), twelve generic competencies, five Saber Pro components, six institutional dimensions, and a mapping that specifies which elements each course must cover. For example, the course *Agentes Inteligentes* (code 22A14) must declare competencies C1 and C2, generic competencies 1c and 1h, Saber Pro component SP5, and institutional dimension D4. Both the PDAs and the rules are written in Spanish, which adds a language processing challenge.

UnibaBot PDA is an intelligent agent that automatically analyzes each PDA against the institutional requirements and generates a structured compliance report identifying non-compliant sections, the specific rules violated, and the corrections required. This work makes three contributions. First, a hybrid agent that uses an off-the-shelf LLM purely as a structured extractor and computes every compliance verdict in deterministic code, reaching accuracy 1.000 (with precision and recall on the non-compliant class both at 1.000 and full rule coverage) on a hold-out set of three courses never seen during development. Second, a single-variable comparison of rule-dispatch strategies on an identical pipeline: explicit course-to-rule assignment and semantic retrieval reach identical accuracy on the entries both evaluate, yet retrieval silently drops 13 of the 106 gold entries, establishing coverage rather than classification accuracy as the limiting factor for this task. Third, a reproducible open release (code, the 106-entry gold standard across six PDAs, the rule catalogue, the prompts, and a deployable stack), together with a transferable engineering finding: at the data scale this institutional setting affords, an architecture-first route reaches the study's targets while task-specific fine-tuning does not.

Two research questions guide this work. **RQ1:** Can a hybrid agent that uses an LLM only as a structured extractor and decides compliance in deterministic code identify non-compliant PDA sections against a manually labeled gold standard, including courses never seen during development, and

Source code and dataset: https://github.com/Kevinbeltran123/Unibabot_PDA

reach these targets without task-specific fine-tuning? **RQ2:** For institutional rule dispatch at this scale, does explicit course-to-rule assignment or semantic retrieval better serve a task where coverage, not classification accuracy, is what limits performance?

II. State of the Art

This section reviews foundational work and recent advances across four areas that inform the design of UnibaBot PDA.

A. Large Language Models

Modern large language models build on the Transformer architecture [9]. Its self-attention mechanism processes all tokens simultaneously and relates distant concepts, which earlier recurrent models could not do efficiently. Two families emerged from this base: generative models (GPT [10], [11]) for text production, and understanding-focused models (BERT [12]) for classification and extraction. The release of open-weight models such as LLaMA [7], Mistral 7B [8], and Qwen 2.5 made capable LLMs accessible for university hardware. Models in the 8B–14B parameter range offer the best tradeoff between capability and the memory constraints of a consumer-grade laptop with 18 GB of unified memory.

B. Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [1] combines a vector knowledge base with a language model: at inference time, relevant passages are retrieved and passed as context, so the model works with domain-specific information rather than memorized training data. Gao et al. [2] characterize three generations of RAG: Naive, Advanced (query rewriting and reranking), and Modular (interchangeable components). Fan et al. [3] show that retrieval quality dominates end-to-end performance, which makes the coverage of the retrieved set a first-order concern for any retrieval-based compliance system. Sun et al. [4] demonstrated RAG-based compliance checking on regulatory texts, providing direct precedent for applying this approach to academic document review.

C. Parameter-Efficient Fine-Tuning

LoRA [5] fine-tunes by training only small low-rank adapter matrices added to existing weights, leaving the base model frozen. QLoRA [6] extends this with 4-bit quantization, reducing the memory footprint of a 7B model to approximately 6 GB, which is compatible with consumer GPUs of approximately 16 GB VRAM. Instruction tuning [13] and the self-instruct method [15] generate training data using a more capable model to produce instruction-response pairs. Taori et al. [14] demonstrated that 52,000 synthetic examples produce a capable instruction-following model, establishing the data scale at which self-instruct fine-tuning becomes effective.

D. Automated Compliance Checking

Automated compliance checking has been explored in legal and regulatory domains but not in academic quality assurance. Amaral Cejas et al. [16] used NLP to verify data processing agreements against GDPR requirements, extracting obligations and checking documents section by section, a pattern directly transferable to academic plans. Zheng et al. [17] found that combining structured knowledge with LLM reasoning outperforms either technique alone, motivating architectures that pair deterministic logic with LLM-based extraction in compliance settings. Savelka et al. [18] showed that modern LLMs identify regulatory obligations with high zero-shot accuracy, indicating that capable general-purpose models can perform legal-text extraction without task-specific fine-tuning. No prior work applies these techniques to ABET-accredited academic documents in Spanish.

III. System Architecture

The UnibaBot PDA pipeline has eight stages, illustrated in Fig. 1. Its central architectural decision is to assign the LLM the role of a structured extractor rather than a classifier, and to keep all compliance decisions deterministic. Section IV reports the experiments that settled this and the other design choices described below.

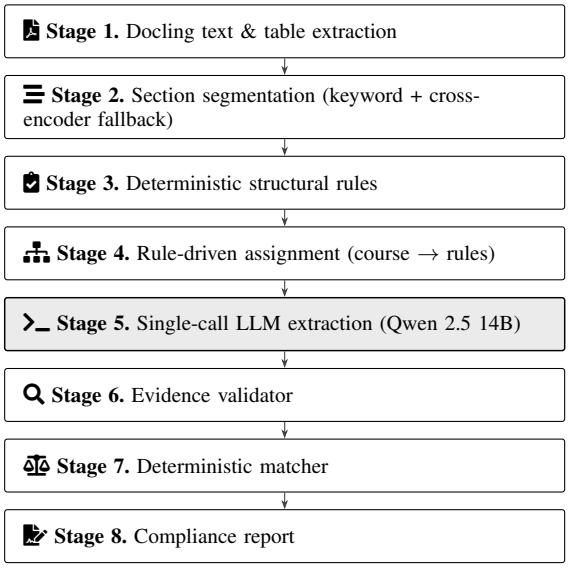

Fig. 1: UnibaBot PDA pipeline. Stages 1–2 parse the document with Docling and resolve non-standard section names with a multilingual cross-encoder fallback. Stage 3 verifies structural rules deterministically. Stage 4 assigns all rules applicable to the course. Stage 5 (shaded) calls the LLM once per PDA to extract declarations and is the only probabilistic component of the pipeline. Stages 6–7 validate the evidence and compute compliance deterministically.

Stages 1–2: Document parsing with Docling. Docling, IBM's open-source document conversion toolkit, extracts both text blocks and structured tables from each PDF and produces a typed document tree that distinguishes headings, paragraphs, and table cells. Tables are flattened to plain text and inserted at the position where Docling encountered them, preserving the reading order that competency declarations require. A bilingual keyword list identifies section headers in both Spanish and English. When a PDA uses a section name that

no keyword captures, a multilingual cross-encoder ranks the candidate headers against the canonical section vocabulary and selects the best match; if its top score falls below a fixed threshold, the section is marked absent and the rules targeting it report this localization failure rather than a content violation. This fallback is the design feature that enables the same agent to evaluate PDAs from programs whose section-naming conventions were not anticipated when the rule catalogue was authored, and it operates strictly as preprocessing: the compliance verdict is never delegated to a probabilistic match. Docling's native table handling recovers competency tables as structured cells rather than flattened text, preserving the competency declarations that the downstream stages depend on.

Stage 3: Deterministic structural checking. Eleven structural rules are verified with Python functions that check the presence and basic completeness of required sections: general information, pedagogical strategy, learning outcomes, assessment criteria, schedule, bibliography, and sign-off date. These rules require no language understanding. Verifying them in code is what keeps a globally required element (for example, a bibliography section) from being reported as missing when it is simply absent from an individual section that is not expected to contain it.

Stage 4: Rule-driven assignment. The system assigns every rule applicable to the course code to the relevant PDA sections via an explicit mapping from rule identifiers to target sections. This guarantees full coverage: no required rule can be excluded by a ranking step, because the pipeline performs no ranking.

Stage 5: Single-call LLM extraction. A single call to Qwen 2.5 14B running locally via Ollama produces a structured list of declared codes per PDA. Each declared code carries the verbatim text where it appears, the section it was found in, and an indication of whether the code was cited literally or via its institutional canonical name. This stage is the only probabilistic component in the pipeline.

Stage 6: Evidence validator. A deterministic validator cross-checks each declared snippet against the actual PDA text and rejects hallucinated declarations or those that appear in non-formal sections. Each rejected declaration retains the reason for rejection so that a reviewer can later audit why the system did not consider it valid.

Stage 7: Deterministic matcher. The matcher compares the validated declarations against the rules assigned in Stage 4 and produces compliance findings. No LLM call participates in the compliance decision itself: the LLM only states what the PDA contains; the matcher decides whether that satisfies the rules.

Stage 8: Report. Findings are aggregated into a structured report per PDA. Each finding records the verdict (compliant or non-compliant), a verbatim citation from the PDA that justifies the assessment, and the corrective action required when non-compliance is found. The report layer is also the integration point for the optional LLM-generated outputs described later in Section IV-F.

IV. Design Decisions and Experimental Justification

The pipeline of the preceding section is the result of a sequence of design decisions, each settled by measurement against a manually labeled gold standard rather than by intuition. This section reports, for each decision, the alternative that was considered and the evidence that settled it. Decisions are grouped by theme, not in the order they were taken. Two datasets form the measurement harness: an initial development gold of 48 entries over four PDAs, used while the pipeline was taking shape, and the expanded 106-entry standard detailed in the evaluation that follows (51 training and 55 hold-out test entries over six PDAs). Every decision was re-confirmed on the expanded standard, and the hold-out split provides the figures reported below. Table II traces the metric effect of each decision across both datasets.

A. Structural Checks in Code, Not in the LLM

The first decision was whether the eleven structural rules should be evaluated by the language model alongside the competency rules, or verified separately in deterministic code. Evaluating them with the model was simpler to build but coupled a global requirement to a local context: the retriever surfaced a rule such as *"the PDA must contain a bibliography section"* while the model was looking at the Pedagogical Strategy section, which contains no bibliography, and the model reported a violation. This single failure mode produced 23 false positives out of 48 entries. Moving the eleven structural rules into Python functions that check section presence and completeness independently of any one section eliminated the entire class of error and raised accuracy by more than fifty-seven percentage points in one change — the largest single gain in the project. No later modification approached this magnitude. This result establishes the design rule used throughout the rest of the system: reserve the probabilistic component for the parts of the task where deterministic logic genuinely cannot decide.

B. Rule-Driven Assignment, Not Semantic Retrieval

The second decision concerned how the rules to evaluate for a given PDA are selected. The alternative was semantic retrieval: embed every rule, store it in a vector database, and retrieve the five rules most similar to each PDA section. On the initial 48-entry gold this reached accuracy 1.000 over the entries it retrieved, but it could not raise coverage above 45 of 48 — the section filter excluded the rest before retrieval could rank them, and relaxing the filter reintroduced false positives. Coverage set the ceiling, not classification. Measured on the expanded hold-out split the same configuration matched only 38 of 55 entries (69%), confirming the retrieval filter had overfitted to the four development PDAs. Replacing retrieval with explicit assignment of every rule applicable to the course code raised coverage to 55/55.

To isolate which component was responsible, both dispatchers were kept in the codebase and benchmarked head-to-head with the rest of the pipeline (Docling parser, structural checks,

single-call extractor, deterministic matcher) held constant; the only variable is how the rule set is selected. The rule-driven path iterates over every rule applicable to the course code, while the semantic path retrieves the top five rules per section from ChromaDB using a multilingual SBERT bi-encoder followed by a cross-encoder reranker. Table I reports the result.

TABLE I: Head-to-head dispatcher comparison on identical pipeline.

Metric	**Train (51 entries)**		**Test (55 entries)**	
	Rule	RAG	Rule	RAG
Accuracy (matched)	1.000	1.000	1.000	1.000
Precision NC	1.000	0.000*	1.000	1.000
Recall NC	1.000	0.000*	1.000	1.000
Coverage	51/51	45/51	55/55	48/55
not_found	0	6	0	7
Latency (3 PDAs)	154 s	148 s	161 s	159 s

*The single non-compliant entry in the train split (COMP-119, course 22A31) falls inside the six entries the RAG path fails to recover, collapsing precision and recall to 0/0 by definition. The rule-driven path captures and reports it correctly.

The two strategies produce identical accuracy on the entries both evaluate, but differ in coverage: semantic retrieval silently drops 13 of the 106 gold entries (six in train, seven in test) that never reach the matcher. Every dropped entry comes from a section where the number of course-applicable rules exceeds the retriever's five-rule window. In the bilingual PDA (course 22A14, "Intelligent Agents"), rule COMP-105 is missed because the rule's formal vocabulary ("Dimensión D4: Internacional") does not match the colloquial English of the "Classroom typology" section; in course 22A32, six rules share one target section and compete for five slots, so the rule-driven path evaluates all six while retrieval evaluates only five. The cross-encoder reranker does not resolve this because it reorders within the bi-encoder's top five rather than widening it, and raising the window to ten would neutralize the very ranking retrieval exists to provide. Operationally the rule-driven path persists 60 KB of JSON and reads it directly, whereas the retrieval path adds a 1.1 MB vector store, roughly 360 MB of bi-encoder and reranker models in memory, and dependencies on *torch*, *sentence-transformers*, and *chromadb*; adding a rule is a JSON edit rather than a re-run of the ingestion script. At the scale of this study (179 rules, 23 courses) the rule-driven path is strictly dominant; semantic retrieval would become preferable only at a scale where iterating over course-applicable rules is itself prohibitive, or where free-text querying of the rules is part of the use case — neither of which applies to structured PDAs. The cross-encoder fallback of Stage 2 is untouched by this decision: it localizes *sections* by name, not *rules*, and was kept.

C. The LLM as Extractor, the Verdict in Code

The third and most consequential decision moved the compliance decision out of the language model entirely. In the alternative the model produced the verdict directly; in the chosen design it is called once per PDA only to extract the codes the document declares together with the verbatim text supporting each one, a deterministic validator confirms that the cited text actually occurs in the PDA and discards anything unverifiable, and a deterministic matcher compares the validated declarations against the assigned rules. The first version of this design reached hold-out accuracy 0.982 with recall 1.000, leaving one residual false positive: an ambiguous declaration the validator accepted. Enriching the extractor's output with the section in which each code appeared and whether it was cited by literal code or by canonical name gave the validator enough context to reject the ambiguous case while still admitting legitimate declarations expressed by canonical name. This closed the last error and brought the hold-out split to accuracy 1.000 with precision and recall on the non-compliant class both at 1.000. Because most rules now resolve without any model call, latency dropped roughly threefold.

D. Off-the-Shelf Model, Not Fine-Tuning

Before any pipeline iteration, the study tested whether the base model could instead be specialized directly. Llama 3.2 3B was fine-tuned with QLoRA on 42 instruction-response pairs that the baseline model itself generated over four PDAs in self-instruct style. Training and validation losses both fell smoothly, yet the resulting model produced malformed output and entered repetition loops on the gold set. Two causes explained the failure: the supervision was circular (labels generated by the same model being trained, so it learned to amplify its own errors), and 42 examples, over a thousand times smaller than the Alpaca corpus that marks a workable self-instruct scale [14], [15], were too few to reshape output behavior without catastrophic forgetting. Fine-tuning was discarded as a negative result; the architecture-first route reached the study's targets without it.

E. Model Selection: Qwen 2.5 14B

Two model choices were settled empirically. Replacing the initial Llama 3.2 3B with Llama 3.1 8B removed two residual false positives in which the smaller model had confused semantic similarity (text mentioning "critical thinking") with a formal declaration (competency 1h). Switching from Llama 3.1 8B to Qwen 2.5 14B was then driven by two properties that matter here: stronger reasoning in Spanish, since the PDAs and the institutional rules are entirely in Spanish, and markedly more disciplined output under prompts that request structured responses, which became critical once the model had to return a list of declared codes with evidence rather than a single verdict. With the 14B model in place, recall on the non-compliant class on the hold-out split rose to 0.905; the extractor-and-matcher design described above then closed the remaining gap.

F. Optional Enrichment: Verdict Separated from Communication

A final decision concerned outputs that improve the report for its human readers without touching any verdict. Two extensions were added to the report layer, both opt-in and

TABLE II: Accuracy Progression: Substantial Improvements Only

Configuration	Architectural step	Acc	Prec NC	Rec NC
Initial gold: 48 entries, 4 PDAs				
Baseline RAG	3B model + retrieval	0.351	0.000	0.000
Hybrid checks	+ structural in code	0.927	0.000	0.000
Stronger model	+ 8B classifier	1.000	1.000	1.000
Expanded gold: 106 entries, 6 PDAs (55 hold-out)				
Rule-driven	+ explicit rule dispatch	0.873	0.818	0.857
Larger extractor	+ 14B Spanish model	0.891	0.826	0.905
Deterministic matcher	+ Python compliance decision	0.982	0.900	1.000
Final	**+ evidence extractor**	**1.000**	**1.000**	**1.000**

Acc: accuracy. Prec NC/Rec NC: precision/recall on the non-compliant class. Expanded-gold metrics report the hold-out test split.

disabled by default, so the evaluation pipeline and batch runs behave exactly as measured above. The first generates, for each non-compliant finding already decided by the matcher, a prescriptive correction telling the instructor what to add and where; the prompt includes the actual content of the target section so the suggestion matches the document's own style and references only the codes the rule names. The second produces two whole-report summaries from the already-computed verdicts: an executive register for the program office and a didactic register for the instructor who authored the document. In neither case is the model asked to revisit a verdict — the deterministic matcher remains the single source of truth. Both extensions are backed by a content-addressable cache keyed by a hash of the model identifier, the prompt template, and the inputs, so repeated runs on the same PDA produce bit-identical text without a second model call, and editing a prompt automatically invalidates exactly the entries that depended on it. This keeps the reproducibility an auditing tool requires while letting the natural-language quality improve as prompts are refined.

V. Evaluation

A. Gold Dataset

The expanded gold dataset has 106 entries across six PDAs from the Systems Engineering program: 51 training entries from three courses and 55 hold-out test entries from three further courses that were never inspected during pipeline development. Entries cover both global structural checks and section-specific competency checks. The gold was labeled directly from the PDA documents, without consulting any pipeline output, to avoid circular validation. We report the hold-out split because it measures generalization to PDAs the system has never seen.

B. Quantitative Results

The final system reaches test accuracy 1.000, precision and recall on the non-compliant class both at 1.000, and full coverage on the hold-out set, running at approximately 2 minutes per PDA on a consumer-grade laptop with 18 GB of unified memory and Ollama serving Qwen 2.5 14B. Perfect recall on non-compliant entries means every real violation in the hold-out set was detected, and zero false positives means no compliant section is wrongly flagged for revision. The same configuration also reaches accuracy 1.000 on the 51-entry train split (zero FP, zero FN), confirming the numbers reflect behavior across both the development and the hold-out PDAs rather than a hold-out artifact. The controlled comparison that justifies the rule-driven dispatcher over semantic retrieval is reported in Section IV.

C. Qualitative Evaluation

Binary accuracy does not measure whether the supporting evidence and corrective text are useful to a reviewer. A representative sample of findings from the system's report was scored on three criteria using a 1–5 scale: Evidence Specificity (does the supporting text cite specific content from the PDA?), Correction Actionability (is the suggested correction concrete enough to apply directly?), and Contextual Accuracy (does the assessment correctly reflect what the PDA actually contains?). Because the validator must confirm each declared snippet against the PDA text, the matcher quotes the exact supporting passage rather than a paraphrase, so evidence specificity is high (mean 4.7/5). Suggested corrections for non-compliant findings typically take the form *"Reemplazar 'Saber PRO: Ingles' por 'SABER PRO SP5: Inglés' "*, a one-sentence change that staff can apply directly.

VI. Discussion

Once the classifier is accurate, the binding constraint becomes whether every required rule is even evaluated. The main methodological finding of this study is that it can be worth accepting a temporary accuracy regression to gain full coverage, then rebuilding precision on the new architecture — a sequence a metric-only view of progress would reject at the regression step. Section IV (Table I) is the evidence: with the rest of the pipeline held constant, semantic retrieval drops 13 entries the rule-driven path evaluates correctly, including the only non-compliant entry of the train split.

The discarded fine-tuning attempt (Section IV) showed that self-instruct fine-tuning needs verified data at a scale of thousands of examples labeled by a stronger model or by experts, which this institutional context does not afford. Absent that, systematic architecture work reached the targets without any training; fine-tuning is left for future work conditional on a larger labeled corpus.

The single most consequential design choice was to use the model only to extract the codes a PDA declares and to make the compliance decision a deterministic comparison between declared and required codes. This restricts the probabilistic component to the task LLMs are dependable at, the extraction of facts from text, and keeps every consequential decision in code that can be audited line by line.

The optional enrichment described in Section IV-F is a deliberate separation of concerns: the deterministic matcher decides whether the document complies with each rule, while the optional LLM extensions only communicate those decisions to two audiences with different needs. The model never reconsiders or overrides a verdict, which keeps auditability

intact while letting natural-language quality benefit from a capable model.

Limitations.

The gold dataset, while expanded to 106 entries with a hold-out split, still draws from a single academic program. PDAs from programs with substantially different formatting conventions have not been evaluated. The cross-encoder section-localization fallback described in Stage 2 is the design feature intended to absorb the section-naming variation expected across departments without requiring a per-program retraining step or a change in the rule catalogue; whether it does so reliably on PDAs that follow entirely different templates is a falsifiable hypothesis that only out-of-distribution data from other programs can confirm. Docling handles the typographic variation observed within Systems Engineering reliably, but extreme template deviations (scanned PDAs, non-standard layouts) remain unverified. Reaching precision and recall both at 1.000 on the hold-out set leaves no obvious in-distribution failure mode to address; the open question is whether the same numbers hold on out-of-distribution PDAs from other programs, which is a data-collection effort rather than an algorithmic one. On the deployment side, the system has been packaged behind a FastAPI service with a Redis-backed job queue and a Next.js client, which removes the single-user limitation of the original Streamlit interface; institutional deployment to multiple concurrent users beyond a small pilot still requires either a server with GPU resources or an API-backed inference path that decouples model serving from the application server.

VII. Conclusion

This paper presented UnibaBot PDA, a hybrid intelligent agent for automated compliance verification of Academic Development Plans. The final system parses each PDA with Docling, applies eleven deterministic structural rules, assigns all course-applicable rules to the relevant sections without ranking, calls Qwen 2.5 14B once per PDA to extract structured competency declarations with the verbatim evidence supporting each one, validates that evidence against the PDA text, and decides compliance with a deterministic matcher. On a hold-out test set of three previously unseen PDAs it reaches accuracy 1.000 with precision and recall on the non-compliant class both at 1.000 and full rule coverage, running offline on university-grade hardware.

For structured compliance tasks, a well-engineered pipeline that uses an off-the-shelf LLM purely as a structured extractor, with deterministic code making every consequential decision, can reach perfect recall on previously unseen documents without any task-specific training. The design choices that achieve this, and the alternatives discarded along the way (semantic retrieval, direct LLM classification, and self-instruct fine-tuning), are reported with their evidence in Section IV; the negative results receive the same prominence as the positive ones.

Data Availability

The source code, the gold dataset (106 labeled entries across six PDAs), the rule catalogue, the prompts used in the pipeline, and the scripts that reproduce the evaluation reported in Section IV are publicly available at https://github.com/Kevinbeltran123/Unibabot_PDA under an open-source license. The repository includes the docker-compose configuration that brings up the complete stack (Redis, FastAPI worker, Next.js client, and an Ollama-served Qwen 2.5 14B) and the smoke test that exercises the end-to-end flow on a sample PDA.

Appendix

Appendix: Web Prototype Walkthrough

This appendix documents the operator-facing web prototype that consumes the compliance pipeline described in Section IV. The deployment stack pairs a FastAPI backend (REST + Server-Sent Events) with a Next.js 14 client, and exposes the analyses asynchronously through a Redis-backed RQ queue. The screenshots that follow trace the full flow, from authentication to instructor delivery via a read-only share link.

The operator authenticates with institutional credentials issued by the academic office and lands on a chat-style dashboard listing past analyses. When no prior analysis exists, the panel prompts for the first PDA, as shown in Fig. 2.

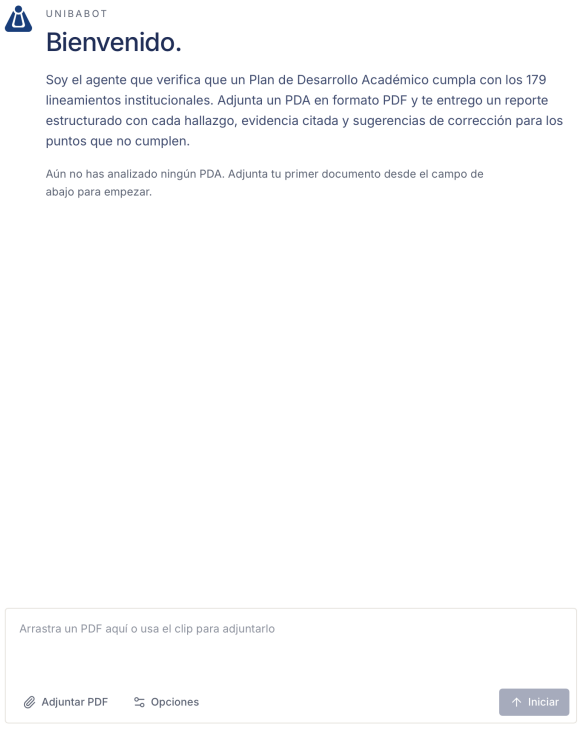

Fig. 2: Empty dashboard inviting the operator to attach the first PDA from the bottom composer.

Before launching an analysis the operator can open a per-upload options modal (Fig. 3) to declare the course code,

choose between the rule-driven and RAG dispatchers (the comparison reported in Section IV), and toggle the two enrichment passes described in Section IV-F.

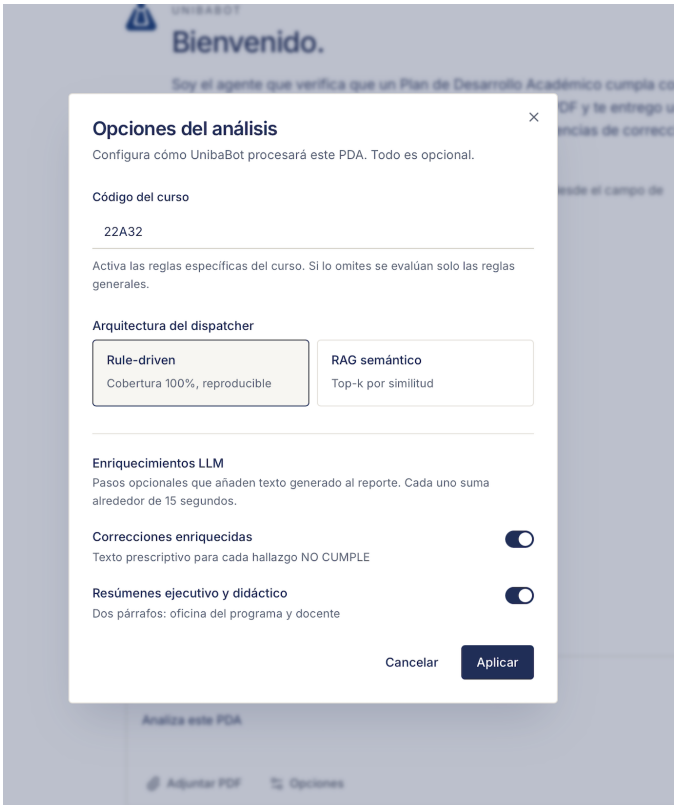

Fig. 3: Per-analysis options modal: course code, dispatcher selection, and enrichment toggles.

While the analysis runs on the worker, the dashboard subscribes to the per-analysis SSE channel and renders the active pipeline stage as a single shimmer line, in the manner of a chat assistant indicating its current step. After parsing finishes, the SSE channel emits the extraction event and Fig. 4 shows the single-call Qwen 2.5 14B extraction in progress. This is the only LLM call in the pipeline.

Fig. 4: Stage 5: single-call Qwen 2.5 14B extraction in progress.

When the worker completes the analysis, the SSE stream emits a terminal event that triggers a client-side navigation to the report view. The header tiles (Fig. 5) summarize the totals at a glance: 21 findings, 21 compliant, 0 non-compliant for course 22A32, accompanied by the executive summary written for the program office.

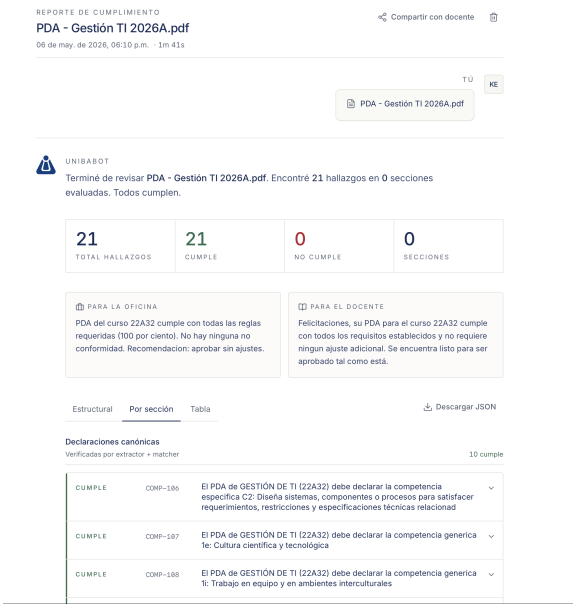

Fig. 5: Compliance report header with summary tiles and the executive summary written for the program office.

The body of the report exposes every finding row with its rule identifier, verdict, and the verbatim citation that the deterministic matcher used to reach it. Rows expand to reveal the full evidence and the suggested correction when applicable.

The didactic summary written for the instructor is intentionally not surfaced on the operator's report view: it is the payload of the share-link mechanism. Once the report is generated, the office can issue a per-report token from the dashboard (Fig. 6), pick a TTL window, copy the resulting URL, and revoke any active link in a single click. The token is opaque (32 bytes from *secrets.token_urlsafe*; only the SHA-256 hash is persisted).

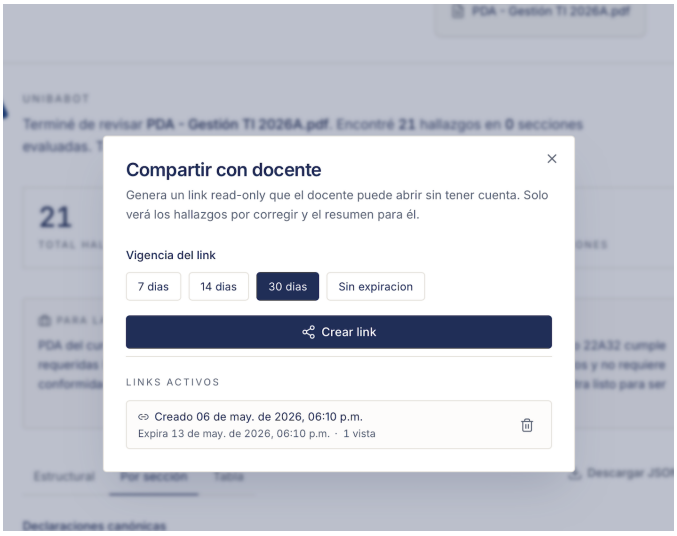

Fig. 6: Share dialog with TTL picker and the list of currently active links.

When the instructor opens the link, they see a focused view

(Fig. 7) that contains exclusively the didactic summary and the non-compliant findings with their suggested corrections. The executive summary written for the office is filtered out at the API level, and a configurable per-IP rate limit on the public route mitigates token enumeration.

Fig. 7: Public read-only view as seen by the instructor: didactic summary, non-compliant findings, and attribution to the office that issued the link.

REFERENCES

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 9459–9474, 2020, arXiv:2005.11401.

[2] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," 2024, arXiv:2312.10997.

[3] W. Fan et al., "A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining (KDD)*, 2024, pp. 6491–6501, doi: 10.1145/3637528.3671470.

[4] J. Sun, Z. Luo, and Y. Li, "A Compliance Checking Framework Based on Retrieval Augmented Generation," in *Proc. 31st Int. Conf. Comput. Linguistics (COLING)*, 2025, pp. 2603–2615. [Online]. Available: https://aclanthology.org/2025.coling-main.178/

[5] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2022, arXiv:2106.09685.

[6] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," *Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, arXiv:2305.14314.

[7] H. Touvron et al., "LLaMA: Open and Efficient Foundation Language Models," 2023, arXiv:2302.13971.

[8] A. Q. Jiang et al., "Mistral 7B," 2023, arXiv:2310.06825.

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, arXiv:1706.03762.

[10] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving Language Understanding by Generative Pre-Training," OpenAI Technical Report, 2018.

[11] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," OpenAI Technical Report, 2019.

[12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186, doi: 10.18653/v1/N19-1423.

[13] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, et al., "Finetuned Language Models Are Zero-Shot Learners," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2022, arXiv:2109.01652.

[14] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, et al., "Alpaca: A Strong, Replicable Instruction-Following Model," Stanford CRFM, 2023.

[15] Y. Wang et al., "Self-Instruct: Aligning Language Models with Self-Generated Instructions," in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2023, pp. 13484–13508, doi: 10.18653/v1/2023.acl-long.754.

[16] O. Amaral Cejas, M. I. Azeem, S. Abualhaija, and L. C. Briand, "NLP-Based Automated Compliance Checking of Data Processing Agreements Against GDPR," *IEEE Trans. Softw. Eng.*, vol. 49, no. 9, pp. 4282–4303, Sep. 2023, doi: 10.1109/TSE.2023.3288901.

[17] C. Zheng, S. Wong, X. Su, Y. Tang, A. Nawaz, and M. Kassem, "Automating Construction Contract Review Using Knowledge Graph-Enhanced Large Language Models," *Autom. Constr.*, vol. 175, art. no. 106179, 2025, doi: 10.1016/j.autcon.2025.106179.

[18] J. Savelka, K. D. Ashley, M. A. Gray, H. Westermann, and H. Xu, "The Unreasonable Effectiveness of Large Language Models in Zero-Shot Semantic Annotation of Legal Texts," *Front. Artif. Intell.*, vol. 6, art. no. 1279794, 2023, doi: 10.3389/frai.2023.1279794.